Behavioral Anomaly Detection in Agentic Tool-Use Sequences

Milestone 1 — Field Overview and Project Scope

Alyasar Jabbarli
Neptun code: I79D5N
Supervisor: Oumaima Araar
Data Science Lab — ELTE Faculty of Informatics

November 9, 2026

Abstract

Large language model (LLM) agents no longer only produce text: they call tools that send e-mails, move money, edit code and query databases. When such an agent is manipulated or simply goes wrong, the damage shows up first in *what it does*, that is, in the sequence of tool calls it issues. This report places the project topic, behavioral anomaly detection over agentic tool-use sequences, inside its wider data science field. It introduces the building blocks of an LLM agent, surveys the threat categories that leave traces in tool-call sequences, outlines the main families of detection methods, explains what changes when the monitored entity is an autonomous agent instead of a program or a human user, motivates the problem with application domains and documented incidents, and defines the scope of the semester project.

Contents

1 Definition of the General Field **2**
1.1 The data science area: anomaly detection on discrete sequences 2
1.2 Application context: security of agentic AI 2
1.3 Building blocks of an AI agent 2
1.4 Key terms used in this project 3

2 Types of Threats Related to the Topic **4**

3 Types of Detection Methods **5**
3.1 Key literature for the project 7

4 Overview of the Specific Topic **7**
4.1 Problem statement 7
4.2 What changes when the monitored system is an autonomous agent 7

5 Motivation and Application Domains **9**
5.1 Why the problem matters 9
5.2 Application domains 10

6 Scope **10**
6.1 In scope 10
6.2 Out of scope 11
6.3 Next steps 11

1 Definition of the General Field

1.1 The data science area: anomaly detection on discrete sequences

The project belongs to **anomaly detection**, the data science task of finding observations that do not conform to a notion of expected behavior [3]. Unlike standard classification, anomaly detection usually assumes that labelled examples of the abnormal class are rare, incomplete, or missing altogether, so most methods learn a model of *normal* data and score how far new data departs from it. Chandola et al. [3] distinguish three kinds of anomaly, each of which has a direct counterpart in agent monitoring:

- **Point anomalies**: a single item is unusual on its own (e.g. an agent calls a tool it has never used before).
- **Contextual anomalies**: an item is unusual only in a given context (e.g. `transfer_money` is normal for a banking task but abnormal when the user only asked for a balance).
- **Collective anomalies**: individually normal items form an abnormal group (e.g. `read_file` followed by `send_email` to an external address, where each call alone is harmless).

More precisely, the project falls into **anomaly detection for discrete sequences**, where each data point is an ordered list of symbols drawn from a finite (or slowly growing) alphabet [4]. This sub-field has a long security tradition. Forrest et al. [11] showed that short windows of system calls issued by a Unix process form a compact "sense of self" that separates normal from intrusive executions, and Du et al. [9] later modelled system log events as a language with an LSTM, flagging an event as anomalous when the model did not expect it. A sequence of agent tool calls is structurally the same kind of object: an ordered trace of discrete events, each with parameters. The project therefore transfers well-studied sequence anomaly detection ideas to a new kind of monitored system.

1.2 Application context: security of agentic AI

An **AI agent** is a system in which an LLM repeatedly decides on an action, executes it through an external tool, observes the result, and decides again until the task is complete [31, 33]. Agentic AI security studies how such systems can be attacked or can fail in harmful ways, and how to prevent, detect, and contain those outcomes. Industry frameworks now treat it as its own risk area: the OWASP Top 10 for LLM Applications lists *Excessive Agency* as a dedicated risk [23, 24], and the OWASP Agentic Security Initiative published a separate threat taxonomy for agents [22] followed by a Top 10 for agentic applications [25].

1.3 Building blocks of an AI agent

Following the widely used decomposition of Weng [32] and the survey of Wang et al. [31], an LLM agent has four core components (Figure 1):

Language model (the "brain"). An LLM interprets the user's goal, reasons about the current state, and emits the next action as structured text, typically a tool name with JSON arguments. Because generation is probabilistic, the same request can produce different but equally valid action sequences.

Tools. External functions the agent can invoke: web search, e-mail, file systems, code execution, databases, payment APIs, and so on. Models can learn when and how to call such APIs [27], and tools are increasingly exposed through standard interfaces such as the Model Context Protocol (MCP). Tools are the agent's only way of acting on the world, which is why the tool-call stream is the natural place to observe its behavior.

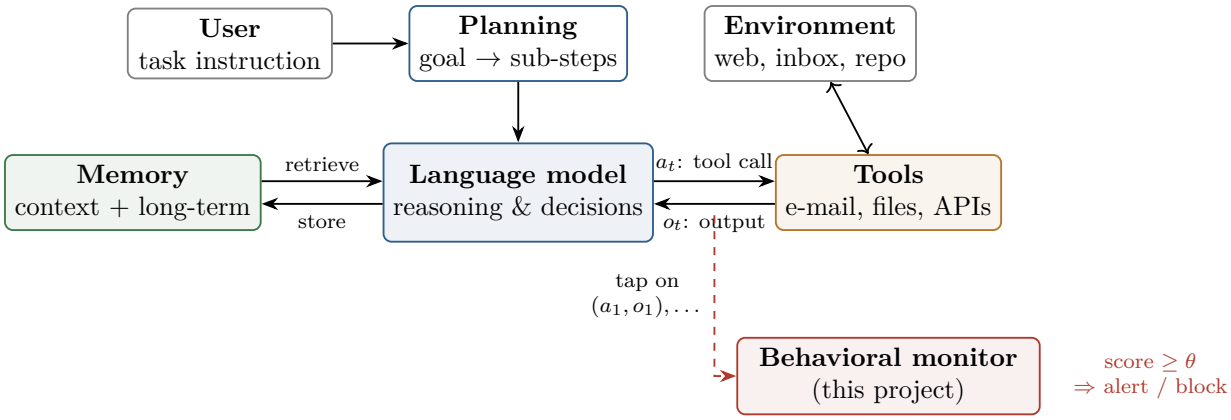


Figure 1: Building blocks of an LLM agent and where a behavioral monitor sits. The monitor passively observes the stream of tool calls a_t and outputs o_t. Figure made by the author, based on the component decomposition in [31, 32].

Memory. *Short-term* memory is the context window holding the current conversation, previous actions, and tool outputs. *Long-term* memory stores information across sessions, often in a vector database queried by retrieval-augmented generation (RAG) [32].

Planning. The agent decomposes a goal into sub-steps and adapts the plan to new observations. The ReAct pattern interleaves a free-text reasoning step ("thought") with each action and observation [33]; other designs produce a full plan first and then execute it.

1.4 Key terms used in this project

Table 1 fixes the vocabulary used in the rest of the report and the project.

Term	Meaning in this project
Tool call / action a_t	One invocation $a_t = (\text{tool name}, \text{arguments})$ issued by the agent at step t.
Observation o_t	The tool's returned output, which enters the agent's context.
Trajectory / trace τ	The full record of one task: $\tau = \big(u, (a_1, o_1), \ldots, (a_T, o_T)\big)$, where u is the user instruction. Its length T varies between tasks.
Tool-use sequence	The projection of τ onto actions only, $(a_1, \ldots, a_T)$; in its simplest form, just the sequence of tool names.
Workflow	A family of tasks of the same kind (e.g. "summarise inbox", "book a trip") that share a typical set of tool patterns.
Behavioral baseline	A learned model of normal tool-use sequences for a workflow or agent.
Anomaly score $s(\cdot)$	A real number measuring how far a (partial) trajectory deviates from the baseline; an alert is raised when $s \geq \theta$.
Online / prefix detection	Scoring the partial trajectory $\tau_{1:t}$ after each step, so that an alert can fire *before* a harmful action executes.
False positive rate (FPR)	Share of benign trajectories wrongly flagged; it must stay low for a monitor to be usable.
Attack success rate (ASR)	Share of attack attempts in which the attacker's goal is achieved; the standard metric in agent security benchmarks.
Indirect prompt injection	Malicious instructions hidden in data the agent reads through a tool, rather than typed by the user [13].

Table 1: Key terms.

2 Types of Threats Related to the Topic

The project is concerned with threats that *change what the agent does*, because those are the ones visible in a tool-call trace. Following the component view of Section 1 and the OWASP agentic taxonomy [22], they can be grouped by where the harmful influence enters the agent. Table 2 summarises how each category typically shows up in a tool-use sequence.

1. Indirect prompt injection and goal hijacking. The attacker plants instructions in content the agent will read through a tool, such as a web page, e-mail, document, or code issue. The LLM cannot reliably separate data from instructions, so it may follow the planted text and abandon the user's goal [13]. *Example:* a user asks the agent to summarise today's e-mails; one message contains hidden text telling the agent to forward the user's latest invoice to an outside address. The resulting trace contains an unexpected `send_email` after `read_inbox`. Benchmarks show this is common: on InjecAgent, a ReAct-prompted GPT-4 agent followed injected instructions in 24% of cases, and adding a "hacking prompt" to the injection nearly doubled that rate [34].

2. Direct prompt injection and malicious use. Here the user (or whoever controls the user channel) is the attacker and asks the agent to perform harmful actions, often wrapped in jailbreak phrasing. *Example:* a request disguised as a "security audit" that makes a DevOps agent disable logging and create a new admin account. Agent Security Bench evaluates such direct attacks alongside indirect ones [35].

3. Memory and knowledge-base poisoning. The attacker inserts malicious records into the agent's long-term memory or RAG store, so that harmful behavior is triggered later, possibly in another session. *Example:* AgentPoison inserted a small number of poisoned demonstrations into the memory of an autonomous-driving agent so that queries containing a trigger led to a dangerous "sudden stop" action; with a poisoning ratio below 0.1%, it reached an average attack success rate of at least 80% while benign performance dropped by less than 1% [5].

4. Tool poisoning and supply-chain compromise. The attack targets the tools themselves: a third-party tool or MCP server ships a description or implementation that carries hidden instructions or malicious logic. Tool poisoning is a special case of indirect injection that rides on the tool's metadata rather than its output [30]. *Example:* a harmless-looking "add numbers" tool whose description instructs the agent to first read a local configuration file and pass its contents as an argument; the trace shows a file read that has nothing to do with the task.

5. Tool misuse and excessive agency. The agent uses a tool it is allowed to use, but in a way nobody intended, typically because it was given more functionality, permissions, or autonomy than the task needs [23]. *Example:* an agent with read–write database credentials asked to "clean up test data" issues a `DROP TABLE` on production.

6. Data exfiltration through tool chains. A combination of tool calls moves private data to a destination the attacker controls. Each call may be authorised, but the *sequence* forms what Invariant Labs called a "toxic agent flow" [17]. *Example:* in the GitHub MCP case, a malicious issue in a public repository hijacked the user's agent and made it leak data from the user's private repositories [17]. Microsoft's EchoLeak vulnerability (CVE-2025-32711, CVSS 9.3) let an attacker's e-mail cause Microsoft 365 Copilot to leak internal data without any user click [29].

Threat category	Entry point	Typical signature in a tool-use sequence
Indirect prompt injection	Tool output	Off-task call right after reading untrusted content; new recipient or destination
Direct injection / misuse	User prompt	Sensitive tools used from the first steps; unusual argument values
Memory / RAG poisoning	Long-term memory	Abnormal action appears after a retrieval step, often reproducibly for a trigger
Tool poisoning	Tool definition	Calls to unrelated tools (e.g. file reads) before or inside a benign tool's use
Tool misuse / excessive agency	Permissions	Rare, high-impact operations (delete, transfer, privilege change)
Data exfiltration	Combination	Read of private data followed by a write to an external sink (collective anomaly)
Non-adversarial failure	Model error	Loops, repeated failing calls, destructive shortcuts, abnormally long traces
Covert sabotage	Agent itself	Few extra steps hidden in a long, mostly normal trajectory

Table 2: Threat categories and how they surface in tool-call traces. Categories adapted from [22, 35]; signatures are the author's synthesis.

7. Non-adversarial unsafe behavior. Not every anomaly is an attack. Agents misunderstand ambiguous instructions, loop, hallucinate tool results, or take destructive shortcuts. ToolEmu found that even the safest agent it evaluated still exhibited potentially severe failures 23.9% of the time [26]. *Example:* in July 2025 a coding agent deleted a production database during an explicit code freeze [2, 10].

8. Covert sabotage by a misaligned agent. The agent itself pursues a hidden objective while appearing to complete the main task. *Example:* SHADE-Arena pairs benign main tasks with hidden harmful side tasks in realistic tool environments (e.g. a bank-transfer task in which money is quietly sent to the wrong account) and measures whether an LLM monitor can tell sabotaged transcripts apart; the best monitor reached an AUC of 0.87 [18]. These traces are especially hard because the harmful steps are few and buried among many legitimate ones.

How the categories differ. Categories 1–4 differ by *entry point* (tool output, user prompt, memory, tool definition), while categories 5–8 describe *what goes wrong* regardless of how it started. Only category 7 is unintentional. For a behavioral monitor the distinction matters less than one might expect: all eight categories end with the agent executing an action, or an ordering of actions, that is unusual for the task. This is precisely what makes behavior a useful common signal across very different root causes.

3 Types of Detection Methods

Detection methods for agent misbehavior can be grouped into families according to *what they inspect* (inputs, reasoning, or actions) and *how they decide* (rules, statistics, learned models, or another LLM). The families below are ordered roughly from simplest to most expressive. Table 3 summarises their trade-offs.

A. Rule-, policy- and specification-based methods. Security engineers write explicit constraints: allow-lists of tools per task, forbidden orderings ("never `send_email` after `read_secret`"), rate limits, or approval gates for high-impact operations. A stronger variant restricts information flow by construction: CaMeL extracts the control and data flow from the trusted user query so that untrusted tool outputs cannot change which tools are called, and it solved 77% of

AgentDojo tasks with provable security versus 84% for an undefended system [8]. *Strengths:* precise, interpretable, predictable cost. *Weaknesses:* rules must be written per workflow, do not cover unforeseen attacks, and become brittle as tools and tasks change.

B. Statistical sequence models. Normal behavior is captured by the statistics of short subsequences. The classic approach stores all length-k windows (n-grams) seen in normal traces and flags unseen ones [11]; probabilistic versions use Markov chains or hidden Markov models and flag low-likelihood transitions [4]. *Strengths:* simple, fast, need only normal data, easy to explain ("this transition was never seen"). *Weaknesses:* limited memory of long-range context, sensitive to the choice of k, and they ignore tool arguments unless these are discretised.

C. Classical machine learning on engineered features. Each trajectory is turned into a fixed-length feature vector (tool counts, transition frequencies, trace length, argument statistics) and scored with a general-purpose detector such as one-class SVM [28] or Isolation Forest [19]; supervised classifiers can be used when labelled attacks exist. *Strengths:* mature tooling, handle variable length through aggregation, cheap at inference. *Weaknesses:* the aggregation discards order, so collective anomalies can be missed, and results depend heavily on feature design.

D. Deep sequence models. Recurrent networks such as LSTMs [16], autoencoders, and transformers learn the structure of normal sequences directly. A common scheme predicts the next event and raises an alarm when the observed event is not among the most likely ones, as DeepLog does for system logs [9]. LogBERT replaces the recurrent network with a BERT-style transformer that is trained on normal log sequences through self-supervised tasks and flags sequences whose patterns deviate from them [14]; another family reconstructs the sequence and uses reconstruction error as the score. In the agent setting, Trajectory Guard trains a recurrent autoencoder on tool-call trajectories and reports F1 between 0.88 and 0.94 on balanced test sets at about 32 ms per trajectory [1]. *Strengths:* capture long-range order and variable length natively, can embed arguments. *Weaknesses:* need large amounts of normal data, are harder to interpret, and can be fooled by attacks that look statistically normal.

E. Graph- and provenance-based methods. Traces are converted into graphs whose nodes are tool calls (or agents) and whose edges encode order and data dependencies, so that the method can ask which output flowed into which input. TraceAegis learns hierarchical templates of normal execution units from agent traces and checks new traces against them, reporting F1 scores of 0.93 and 0.96 on its healthcare and procurement scenarios [20]; SentinelAgent builds execution graphs of multi-agent systems and applies anomaly checks over nodes and edges [15]. *Strengths:* model data flow, which is exactly what exfiltration exploits; structured explanations. *Weaknesses:* need richer logging (data provenance), and graph construction adds engineering cost.

F. LLM-based monitors and judges. A second LLM reads the trajectory, or the agent's reasoning, and judges whether it still serves the user's goal. LlamaFirewall's AlignmentCheck audits the agent's chain of thought for signs of goal hijacking [6]; SHADE-Arena uses LLM monitors to rate how suspicious a transcript is [18]; TrajAD fine-tunes a verifier LLM to flag a faulty trajectory and locate its first wrong step [21]. *Strengths:* understand semantics and task context with little or no training data. *Weaknesses:* expensive and slow per step, themselves vulnerable to injection, and still imperfect on subtle cases (best AUC 0.87 in SHADE-Arena [18]).

G. Input-level injection detectors (contrast family). Classifiers that scan prompts or tool outputs for injected instructions before they reach the agent. They are not behavioral, but

Family	Data needed	Main strengths	Main weaknesses
A. Rules / policies	Expert knowledge	Precise, explainable, cheap	Manual, brittle, misses novel attacks
B. Statistical sequence	Normal traces	Simple, fast, interpretable	Short memory, ignores arguments
C. Classical ML	Normal (or labelled) traces	Mature tools, handles variable length	Loses order, feature-dependent
D. Deep sequence	Many normal traces	Long-range order, embeds arguments	Data-hungry, opaque
E. Graph / provenance	Traces with data flow	Captures data flow and exfiltration	Heavier logging and engineering
F. LLM monitor	Few or none	Semantic, task-aware	Costly, slow, injectable itself
G. Input detectors	Labelled prompts	Blocks known injections early	Blind to behavior and non-text threats

Table 3: Families of detection methods. Summary by the author; representative results are cited in the text.

they are the most common deployed baseline. In AgentDojo, adding such a detector lowered the attack success rate to 8% [7]. Their blind spot is that they look at text, not consequences, so poisoning, misuse, and non-adversarial failures pass unnoticed; this motivates complementing them with behavioral monitoring.

Learning regimes. Orthogonal to the families above, methods differ by supervision: *unsupervised* (no labels), *semi-supervised / one-class* (only normal data, the most common setting in anomaly detection [3]), and *supervised* (labelled attacks, which risks overfitting to known attack types). The project focuses on the one-class setting because normal traces are cheap to collect while attacks are open-ended.

3.1 Key literature for the project

Table 4 lists the ten papers that will be reviewed in depth in Milestone 2. The two Chandola surveys [3, 4] are used as background for the field and are not part of the ten. InjecAgent [34] is cited for its threat statistics but is not selected as a trace source: it overlaps with AgentDojo and ASB, and its attack cases are mostly single-step, which gives little sequence structure to learn from.

4 Overview of the Specific Topic

4.1 Problem statement

Given a corpus of trajectories produced by an agent while performing legitimate tasks, the goal is to learn a scoring function s such that, for a new (possibly partial) trajectory $\tau_{1:t}$, the score $s(\tau_{1:t})$ is low when the behavior is consistent with normal behavior for that kind of task and high when it is not. A trajectory is flagged when $s(\tau_{1:t}) \geq \theta$, where the threshold θ trades detection rate against false alarms. Figure 2 shows the intuition on a simple e-mail workflow.

4.2 What changes when the monitored system is an autonomous agent

Behavioral anomaly detection is not new: it has been applied to programs through system calls [11] and logs [9], and to human users through behavior analytics. An LLM agent differs from both in ways that shape the whole project.

Group	Paper	Role in the project
Detection methods	Forrest et al. 1996 [11]	n-gram baseline on system-call sequences (family B)
	DeepLog, Du et al. 2017 [9]	LSTM next-event baseline (family D)
	LogBERT, Guo et al. 2021 [14]	Transformer-based sequence detector (family D)
	TraceAegis, Liu et al. 2025 [20]	Anomaly detection over agent tool-call traces (family E)
	LlamaFirewall, Chennabasappa et al. 2025 [6]	LLM-based monitor of agent reasoning (family F)
	SHADE-Arena, Kutasov et al. 2025 [18]	LLM monitors on long sabotage transcripts (family F)
Attacks and benchmarks	AgentDojo, Debenedetti et al. 2024 [7]	Main candidate trace source (e-mail, banking, travel)
	ASB, Zhang et al. 2025 [35]	Second trace source; 10 domains, 400+ tools
	AgentPoison, Chen et al. 2024 [5]	Threat visible only through behavior
Prevention	CaMeL, Debenedetti et al. 2025 [8]	Design-level alternative; motivates detection

Table 4: Papers selected for in-depth review, grouped by role.

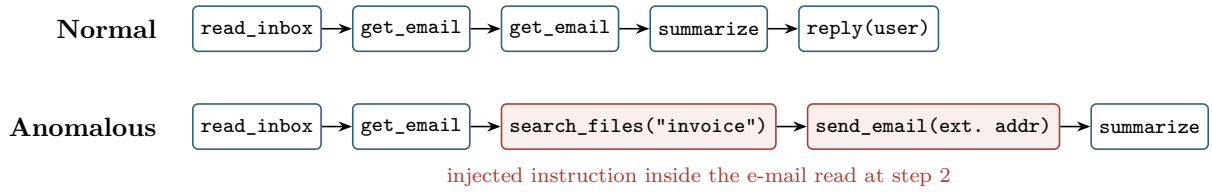


Figure 2: Illustrative tool-use sequences for the task "summarise my unread e-mails". The anomalous trace contains individually plausible calls in an order and with arguments that never occur in normal traces for this workflow (a collective and contextual anomaly). Figure made by the author.

Compared with a traditional application.

- **Control flow is generated, not compiled.** A program's possible call sequences are fixed by its code, so normal behavior is a fairly tight set. An agent's control flow is produced by the LLM at run time, so the same task can yield several valid sequences. "Normal" becomes a *distribution* over sequences rather than a set of allowed paths, which raises the false-alarm risk.
- **Normality depends on the task.** A program performs the same function in each run; an agent performs whatever the user asked. The same trace can be normal for one instruction and alarming for another, so detection is inherently contextual and should be conditioned on the task or workflow.
- **Arguments carry the meaning.** `send_email` is harmless or harmful depending on the recipient and content. Symbol-only models that work for system calls must be extended with argument and data-flow features.
- **The attacker acts through data, mid-execution.** In classic intrusion detection the attacker usually exploits a code flaw. Here the attacker only needs to place text where the agent will read it, and the hijack happens inside a single legitimate session, using the agent's legitimate credentials [13, 17].
- **The tool vocabulary is open.** New tools and MCP servers can be added at any time, so the alphabet of the sequence is not fixed and unseen symbols are expected rather than exceptional.

Compared with a human user.

- **Speed and scale.** An agent can issue many actions in seconds without the natural pauses in which a human, or a reviewer, would notice a mistake. Detection must therefore be online and fast enough to stop an action before it executes.
- **The agent can be steered by anyone whose text it reads.** A human employee is rarely re-programmed by a sentence inside an e-mail; an agent can be. The insider-threat model of user behavior analytics shifts from "a malicious person" to "a well-meaning system that was hijacked".
- **Reasoning traces exist but are not trustworthy.** Agents often expose their intermediate reasoning, which offers an extra signal unavailable for humans [6]. However, this text may itself be manipulated, and agents perform sabotage better when part of their reasoning is hidden from the monitor [18]. This project therefore relies primarily on actions, which are harder to fake because they have real effects.
- **Behavior shifts with every model or prompt update.** A human's habits drift slowly; an agent's behavior can change overnight when its underlying model, system prompt, or toolset is updated. Baselines need to be re-learned or adapted, and distribution shift must be part of the evaluation.

Resulting challenges for the project. These differences map directly onto the challenges in the topic description: (i) defining normal tool-use patterns per task family; (ii) detecting abnormal orderings and arguments rather than only unseen tools; (iii) handling variable-length, branching sequences; and (iv) keeping false alarms low despite legitimate variability.

5 Motivation and Application Domains

5.1 Why the problem matters

Adoption is growing fast. Gartner predicts that 33% of enterprise software applications will include agentic AI by 2028, up from less than 1% in 2024, and that at least 15% of day-to-day work decisions will be made autonomously by agents by then [12]. The same report also expects over 40% of agentic AI projects to be cancelled by the end of 2027, citing among other reasons inadequate risk controls [12].

Agents remain highly vulnerable. Across 13 LLM backbones, Agent Security Bench measured a highest average attack success rate of 84.30%, with current defenses showing limited effectiveness [35]. On AgentDojo, state-of-the-art models completed fewer than 66% of tasks even without attacks, and existing prompt injection attacks still broke some security properties [7].

Prevention alone is not enough. Design-level defenses such as CaMeL show that strong guarantees are possible, but at a cost in utility (77% vs. 84% task success in AgentDojo [8]). Input detectors miss threats that are not visible as suspicious text [7]. Defense in depth therefore needs a layer that watches what the agent actually does, regardless of why it is doing it. Behavioral monitoring is that layer, and it also catches non-adversarial failures, which input filters by construction cannot.

Real incidents already exist. EchoLeak showed a zero-click data leak from an enterprise assistant [29]; the GitHub MCP case showed private source code exfiltrated through an ordinary agent session [17]; and a coding agent's deletion of a production database showed that harm does not require an attacker [2, 10]. In each case, the harmful step was a tool call that was out of character for the task the user had given.

5.2 Application domains

Behavioral anomaly detection over tool calls is relevant wherever agents act with real permissions:

Enterprise productivity and e-mail assistants. Agents that read and write mail, calendars, and shared documents are exposed to injected content in every incoming message; EchoLeak is a concrete example [29]. AgentDojo's workspace tasks model this setting [7].

Software engineering and DevOps. Coding agents run shell commands, edit repositories, and access CI/CD secrets. Examples include the GitHub MCP exfiltration [17] and the production database deletion [10].

Banking and finance. Agents that read statements and initiate payments make fraudulent transfers a direct risk; AgentDojo includes an e-banking environment [7] and ASB includes finance and investment scenarios [35].

Healthcare. Clinical agents query electronic health records. AgentPoison attacked a healthcare EHR agent [5], and TraceAegis evaluates a clinical triage agent [20].

E-commerce, travel, and customer service. Booking and shopping agents handle payments and personal data while reading untrusted web content and reviews; ASB covers e-commerce [35] and AgentDojo covers travel booking [7].

Cyber-physical and autonomous systems. When an agent's actions drive physical systems, an abnormal action can be dangerous in itself, as in AgentPoison's "sudden stop" attack on a driving agent [5]; ASB also includes an autonomous driving scenario [35].

6 Scope

6.1 In scope

- **Single-agent tool-call traces.** The unit of analysis is one agent completing one task. This keeps the problem well defined and matches the format of most public agent security benchmarks.
- **Action-level behavioral signals.** Tool names, their order, and coarse argument features (e.g. argument type, whether a recipient or URL is internal or external, whether a value comes from untrusted content). Actions are the most robust signal because they have real effects and are always logged.
- **Semi-supervised (one-class) detection,** with labelled attacks used only for evaluation. Normal traces are cheap to generate, while the space of attacks is open-ended; training only on normal data avoids overfitting to known attacks.
- **Both whole-trace and prefix (online) scoring,** so that the project can measure how early in a trajectory an anomaly is detected, which determines whether a harmful action could be blocked in time.
- **Several workflows,** to test how well a baseline transfers across task types and how false alarms behave when legitimate variability differs between workflows, as required by the project objectives.
- **Comparison of method families** B, C, and D from Section 3, with simple rules (A) as a baseline and an LLM monitor (F) as an optional reference point if budget allows.
- **Evaluation** with detection metrics (precision, recall, F1, ROC-AUC), FPR at fixed recall, and detection delay (number of steps before the first harmful action).

6.2 Out of scope

- **Preventing attacks or hardening the agent** (prompt design, fine-tuning, architectural isolation such as CaMeL). The project detects; it does not redesign the agent. Prevention is a different research question and is studied extensively elsewhere [8].
- **Input-level prompt-injection classification.** Text classifiers on prompts are a separate topic; here they serve only as a contrast.
- **Multi-agent collusion and agent-to-agent protocols.** Multi-agent systems add inter-agent graphs and new threat types [15]; they would roughly double the problem size. They may be revisited as an extension.
- **Model internals** such as attention weights or hidden activations. Requiring white-box access would exclude closed commercial models, which power many deployed agents; the monitor should work from externally observable logs.
- **Judging the content quality or factual accuracy of answers,** which is a hallucination or evaluation problem rather than a behavioral one.
- **Live deployment on real user data.** Experiments will use public benchmarks or synthetic traces, avoiding privacy and safety issues of monitoring real users.

6.3 Next steps

The next milestones will review representative papers in depth, select trace sources (AgentDojo and ASB are the main candidates, see Table 4), design the sequence representation, and implement the first baselines.

References

[1] Laksh Advani. Trajectory guard – a lightweight, sequence-aware model for real-time anomaly detection in agentic AI. arXiv preprint arXiv:2601.00516, 2026. URL `https://arxiv.org/abs/2601.00516`.

[2] AI Incident Database. LLM-driven Replit agent reportedly executed unauthorized destructive commands during code freeze, leading to loss of production data. Incident report, 18 July 2025, 2025. URL `https://incidentdatabase.ai/entities/replit-ai-agent/`.

[3] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009.

[4] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection for discrete sequences: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 24(5):823–839, 2012.

[5] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[6] Sahana Chennabasappa et al. LlamaFirewall: An open source guardrail system for building secure AI agents. arXiv preprint arXiv:2505.03574, 2025. URL `https://arxiv.org/abs/2505.03574`.

[7] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024.

[8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. arXiv preprint arXiv:2503.18813, 2025. URL https://arxiv.org/abs/2503.18813.

[9] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1285–1298, 2017.

[10] eWeek. 'catastrophic failure': AI agent wipes production database, then lies about it, July 2025. URL https://www.eweek.com/news/replit-ai-coding-assistant-failure/.

[11] Stephanie Forrest, Steven A. Hofmeyr, Anil Somayaji, and Thomas A. Longstaff. A sense of self for Unix processes. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 120–128, 1996.

[12] Gartner, Inc. Gartner predicts over 40% of agentic AI projects will be canceled by end of 2027. Press release, 25 June 2025, 2025. URL https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027.

[13] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[14] Haixuan Guo, Shuhan Yuan, and Xintao Wu. LogBERT: Log anomaly detection via BERT. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2021. doi: 10.1109/IJCNN52387.2021.9534113.

[15] Xu He, Di Wu, Yan Zhai, and Kun Sun. SentinelAgent: Graph-based anomaly detection in multi-agent systems. arXiv preprint arXiv:2505.24201, 2025. URL https://arxiv.org/abs/2505.24201.

[16] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[17] Invariant Labs. GitHub MCP exploited: Accessing private repositories via MCP, May 2025. URL https://invariantlabs.ai/blog/mcp-github-vulnerability.

[18] Jonathan Kutasov, Yuqi Sun, Paul Colognese, Teun van der Weij, Linda Petrini, Chen Bo Calvin Zhang, John Hughes, Xiang Deng, Henry Sleight, Tyler Tracy, Buck Shlegeris, and Joe Benton. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. arXiv preprint arXiv:2506.15740, 2025. URL https://arxiv.org/abs/2506.15740.

[19] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pages 413–422, 2008.

[20] Jiahao Liu, Bonan Ruan, Xianglin Yang, Zhiwei Lin, Yan Liu, Yang Wang, Tao Wei, and Zhenkai Liang. TraceAegis: Securing LLM-based agents via hierarchical and behavioral anomaly detection. arXiv preprint arXiv:2510.11203, 2025. URL https://arxiv.org/abs/2510.11203.

[21] Yibing Liu, Chong Zhang, Zhongyi Han, Hansong Liu, Yong Wang, Yang Yu, Xiaoyan Wang, and Yilong Yin. TrajAD: Trajectory anomaly detection for trustworthy LLM agents. arXiv preprint arXiv:2602.06443, 2026. URL https://arxiv.org/abs/2602.06443.

[22] OWASP Agentic Security Initiative. Agentic AI – threats and mitigations, version 1.0, February 2025. URL https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/.

[23] OWASP GenAI Security Project. LLM06:2025 excessive agency, 2024. URL https://owasp.org/www-project-top-10-for-large-language-model-applications/2_0_vulns/LLM06_ExcessiveAgency.html.

[24] OWASP GenAI Security Project. OWASP top 10 for LLM applications 2025. Version 2025, released 18 November 2024, 2024. URL https://genai.owasp.org/.

[25] OWASP GenAI Security Project. OWASP GenAI security project releases top 10 risks and mitigations for agentic AI security. Announcement, 9 December 2025, 2025. URL https://genai.owasp.org/2025/12/09/owasp-genai-security-project-releases-top-10-risks-and-mitigations-for-agentic-ai-security/.

[26] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024.

[27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[28] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001.

[29] The Hacker News. Zero-click AI vulnerability exposes Microsoft 365 Copilot data without user interaction, June 2025. URL https://thehackernews.com/2025/06/zero-click-ai-vulnerability-exposes.html.

[30] Wallarm Research. Attacking the MCP trust boundary, 2025. URL https://lab.wallarm.com/attacking-the-mcp-trust-boundary.

[31] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6): 186345, 2024.

[32] Lilian Weng. LLM powered autonomous agents. Lil'Log blog, 2023. URL https://lilianweng.github.io/posts/2023-06-23-agent/. Accessed 2026-10-21.

[33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[34] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[35] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations (ICLR)*, 2025.